

20 archivos 16 los escriben ustedes 3 clases abstractas 5 interfaces 40 pruebas automáticas

Vamos a construir el sistema de un delivery, pero cada grupo escribe una parte distinta del mismo programa. El profesor entrega la clase raíz y los dos `main`. Lo demás es de ustedes: cinco interfaces y once clases. Al final de la sesión compilamos todo junto, en pantalla, y vemos si corre.

CÓMO VIENEN LOS ARCHIVOS

El Javadoc es la especificación. Sus archivos no traen instrucciones sueltas: traen el Javadoc completo de cada método — qué hace, qué recibe, qué devuelve, qué excepción lanza — y el cuerpo vacío. Ustedes leen el contrato y escriben el código que lo cumple.

```
/**
 * El precio segun el tamano y los ingredientes extra.
 *
 * <p>FAMILIAR multiplica por 1.6, MEDIANA por 1.0 y PERSONAL por 0.7.
 * Sobre eso se suman $700 por cada ingrediente extra.</p>
 *
 * @return el precio final en pesos
 */
@Override
public double calcularPrecio() {
    // TODO G1
}
```

Las cinco interfaces van un paso más allá: **llegan vacías**. Traen escrito en su Javadoc qué métodos debe exigir el contrato, con qué tipos y qué debe lanzar, y el grupo dueño lo traduce a Java. Es lo primero que se hace hoy.

LA JERARQUÍA COMPLETA

```
Producto (abstracta, la entrega el profesor)
├ Pizza, Sushi      G1  implements Calificable (G5) + Preparable (propia)
├ Bebida            G2  implements Preparable (G1)
└ Combo             G2  implements Preparable (G1) + Descontable (propia)
                        y ademas CONTIENE Productos (composicion)

Pedido              G3  implements Descontable (G2)   contiene Producto[]

MedioPago (abstracta) G4  implements Facturable (G3)
├ Tarjeta          G4
└ Efectivo         G4

Repartidor (abstracta) G5  implements Cobrable (G4) + Calificable (propia)
├ RepartidorMoto   G5
└ RepartidorBici   G5
```

EL ANILLO DE CONTRATOS

Cada grupo es dueño de una interfaz y está obligado a cumplir la del vecino. El círculo se cierra: si un grupo no entrega, no compila nadie.

CONTRATO	LO ESCRIBE		LO CUMPLE
Preparable	G1 Cocina	→	G2 en Bebida y Combo
Descontable	G2 Bebidas	→	G3 en Pedido
Facturable	G3 Pedido	→	G4 en MedioPago

Cobrible	G4 Pagos	→	G5 en Repartidor
Calificable	G5 Reparto	→	G1 en Pizza y Sushi

LAS SEIS REGLAS

- 1. **Primero el contrato, después el código.** Nadie escribe el cuerpo de un método antes del minuto 40. Mientras una interfaz esté vacía, el grupo que la implementa no compila.
- 2. **Cada grupo toca solo sus archivos.** Si necesitan que otro cambie algo, se lo piden a ellos. No se edita el archivo de otro grupo.
- 3. **Nadie modifica** `Producto.java` , `Verificador.java` , `MainDemo.java` ni `MainPruebas.java` .
- 4. **Las firmas no se cambian sin avisar.** Si tocan un método de su interfaz, el grupo que la implementa deja de compilar en ese mismo segundo.
- 5. **Los atributos van `private`** y se validan en el constructor o el setter, lanzando `IllegalArgumentException` .
- 6. **Compilen temprano y seguido** con la línea de `javac` de su ficha. No esperen al final.
- 7. **Truco:** un método que declara `return` y no lo tiene **no compila**. Mientras trabajan, dejen un `return 0;` provisorio para poder ir compilando.

LOS TRES ERRORES QUE VAN A VER

DICE EL COMPILADOR	QUÉ PASÓ
method does not override...	La interfaz que deben cumplir todavía está vacía . No es problema de ustedes: es el grupo dueño el que no la ha escrito.
missing return statement	El método está vacío. Ese es su trabajo. Para ir compilando a medias, un <code>return 0;</code> provisorio.
constructor X cannot be applied...	El constructor está vacío y Java intenta llamar solo a <code>super()</code> . Falta el <code>this(...)</code> o el <code>super(...)</code> con sus parámetros.

CÓMO CORRE ESTO

```
javac *.java           # los 20 archivos juntos. Solo compila si los 5 terminaron.
java MainDemo          # la demo narrativa que se proyecta
java MainPruebas       # el banco de pruebas: 40 líneas [OK] / [FALLA]
```

LOS 120 MINUTOS

0-15	Presentación del caso. Compilamos con todo vacío y falla: esa es la foto inicial.
15-25	Reparto de fichas. Cada grupo lee la suya.
25-40	Ronda de contratos. Cada grupo escribe su interfaz —son dos métodos— y sale a explicársela al grupo que debe cumplirla. Nadie escribe cuerpos todavía.
40-70	Implementación. Cada grupo compila lo suyo por separado.
70-80	Checkpoint proyectado. Los 5 grupos muestran su compilación.
80-100	Integración: todo en una carpeta, <code>javac *.java</code> en vivo.
100-115	<code>MainDemo</code> y <code>MainPruebas</code> . Leemos el marcador.
115-120	Cierre.

Lo que se evalúa: nada. Esta sesión es formativa. La única nota que cuenta es la del marcador de `MainPruebas` , y es colectiva: los 40 [OK] o no salen.

estación HORNO · FRÍO 2 clases 4 sobrecargas hereda de Producto

Ustedes hacen la comida. Dos clases que heredan de `Producto` y que calculan su precio de maneras completamente distintas: eso es lo que después le permite al Pedido sumarlas sin preguntar cuál es cuál.

LOS DOS CONTRATOS

USTEDES LO ESCRIBEN

Preparable

```
int minutosPreparacion()
```

```
String estacionCocina()
```

Está obligado a cumplirlo el **Grupo 2**, en Bebida y Combo.

Escríbanla ustedes en `Preparable.java`: llega vacía, con la especificación en su Javadoc.

DEBEN CUMPLIRLO

Calificable

```
void calificar(int estrellas)
```

```
double promedioEstrellas()
```

Lo escribe el **Grupo 5**. Va en las dos clases: `implements`

`Calificable`.

LO QUE TIENE QUE HACER CADA MÉTODO

MÉTODO	REGLA
<code>Pizza(nombre, precio)</code>	Sobrecarga corta: asume tamaño <code>"MEDIANA"</code> . Llámense al otro constructor con <code>this(...)</code> , no copien el código.
<code>Pizza(..., tamano)</code>	Guarda el tamaño y deja los ingredientes extra en 0.
<code>agregarIngrediente()</code>	Agrega uno. Reutiliza la otra versión.
<code>agregarIngrediente(int)</code>	Agrega varios. Cantidad negativa → <code>IllegalArgumentException</code> .
<code>Pizza.calcularPrecio()</code>	FAMILIAR $\times 1.6$ · MEDIANA $\times 1.0$ · PERSONAL $\times 0.7$, sobre el precio base, más \$700 por cada ingrediente extra.
<code>Sushi.calcularPrecio()</code>	$\text{precioBase} + (\text{piezas} - 10) \times 350$, y \$900 más si lleva palta. Un roll de 10 piezas cuesta exactamente el precio base.
<code>calificar(int)</code>	Solo de 1 a 5. Fuera de rango → <code>IllegalArgumentException</code> . Guarden suma y cantidad de votos.
<code>promedioEstrellas()</code>	Suma ÷ votos. Sin votos todavía, devuelve 0 (ojo con dividir por cero).
<code>minutosPreparacion()</code>	Pizza: 12 + 1 por ingrediente extra. Sushi: 8 siempre.
<code>estacionCocina()</code>	Pizza: <code>"HORNO"</code> . Sushi: <code>"FRÍO"</code> .

PARA COMPILAR SOLO LO SUYO

```
javac Producto.java Preparable.java Calificable.java Pizza.java Sushi.java
```

CÓMO SABEN QUE QUEDÓ BIEN

- [OK] `Pizza` extends `Producto` — sección B
- [OK] G1 cumple `Calificable`, contrato del G5 — sección C
- [OK] El constructor corto asume `MEDIANA` — sección E
- [OK] `calificar(9)` rechazado: solo 1 a 5 — sección F

Si terminan antes: habrán notado que el código de `calificar()` y `promedioEstrellas()` quedó escrito **dos veces**, igual en `Pizza` y en `Sushi`. Prepárense para responder esto en el cierre: ¿dónde debería vivir ese código para escribirlo una sola vez, y por qué en este diseño no se puede? Es la pregunta más interesante del taller.

estación **BARRA** · **ARMADO** 2 clases composición **instanceof**

Ustedes tienen la clase más interesante del taller. El **Combo** es un Producto y además **tiene** Productos adentro: cuando calcula su precio recorre lo que le metieron y le pregunta a cada uno cuánto vale, sin saber si es una pizza, un sushi o una bebida. Polimorfismo dentro de su propia clase.

LOS DOS CONTRATOS

USTEDES LO ESCRIBEN

Descontable

```
double aplicarDescuento(double pct)
boolean tieneDescuento()
```

Está obligado a cumplirlo el **Grupo 3**, en Pedido. **Escribanla** ustedes en **Descontable.java**: llega vacía, con la especificación en su Javadoc.

DEBEN CUMPLIRLO

Preparable

```
int minutosPreparacion()
String estacionCocina()
```

Lo escribe el **Grupo 1**. Va en **Bebida** y en **Combo**.

LO QUE TIENE QUE HACER CADA MÉTODO

MÉTODO	REGLA
Bebida(n, precio, ml)	Sobrecarga corta: sin hielo. Usen <code>this(...)</code> .
Bebida(..., conHielo)	Mililitros $\leq 0 \rightarrow$ <code>IllegalArgumentException</code> .
Bebida.calcularPrecio()	El precio base, más \$500 si pasa de 500 cc.
Bebida.minutosPreparacion()	2 si lleva hielo, 1 si no. Estación: <code>"BARRA"</code> .
Combo(nombre)	Arreglo de capacidad 6, cantidad en 0, descuento inicial <code>DESCUENTO_COMBO</code> .
agregar(Producto)	Si ya no cabe $\rightarrow$ <code>IllegalStateException</code> .
Combo.calcularPrecio()	El corazón del ejercicio. Recorren los items sumando <code>calcularPrecio()</code> y aplican el descuento. Sin un solo <code>if</code> preguntando de qué clase es cada uno.
Combo.minutosPreparacion()	El mayor de los items que sean <code>Preparable</code> , más 3 de armado. No todos lo son: usen <code>if (items[i] instanceof Preparable)</code> . Estación: <code>"ARMADO"</code> .
aplicarDescuento(pct)	Fuera de 0-100 $\rightarrow$ <code>IllegalArgumentException</code> . Lo guarda y devuelve el precio nuevo.
tieneDescuento()	<code>true</code> si el porcentaje es mayor que 0.

PARA COMPILAR SOLO LO SUYO

```
javac Producto.java Preparable.java Descontable.java Bebida.java Combo.java
```

CÓMO SABEN QUE QUEDÓ BIEN

- [OK] G2 cumple `Preparable`, contrato del G1 (Bebida) — sección C
- [OK] G2 cumple `Preparable`, contrato del G1 (Combo) — sección C
- [OK] El for polimorfico suma las 4 clases — sección D
- [OK] Precio negativo rechazado — sección F

Si terminan antes: hagan que un Combo pueda contener otro Combo y prueben qué pasa. ¿Se rompe algo? ¿Por qué el código de `calcularPrecio()` no necesitó cambiar ni una línea?

1 clase, la más grande 3 calcularTotal() 2 agregar() static + final

Ustedes escriben la única clase que **no** hereda de `Producto`, y la que usa el trabajo de todos los demás. En este archivo viven cuatro conceptos a la vez: composición, polimorfismo, sobrecarga y `static`. Es la clase más grande del taller: repártanse los métodos entre ustedes.

LOS DOS CONTRATOS

USTEDES LO ESCRIBEN

Facturable

```
String detalleBoleta()
double totalConIVA()
```

Está obligado a cumplirlo el **Grupo 4**, en `MedioPago`.

Escribanla ustedes en `Facturable.java`: llega vacía, con la especificación en su Javadoc. Ojo con la decisión de diseño: la boleta **no** la emite el pedido, la emite el medio de pago.

DEBEN CUMPLIRLO

Descontable

```
double aplicarDescuento(double pct)
boolean tieneDescuento()
```

Lo escribe el **Grupo 2**.

LO QUE TIENE QUE HACER CADA MÉTODO

MÉTODO	REGLA
<code>Pedido(cliente)</code>	Cliente vacío o <code>null</code> → <code>IllegalArgumentException</code> . Sube el contador <code>static</code> y lo usa como número de este pedido. Arreglo de capacidad 20. Ojo: <code>numero</code> es <code>final</code> , hay que asignarlo sí o sí acá.
<code>agregar(Producto)</code>	Sobrecarga corta. Reutilicen la otra.
<code>agregar(Producto, int)</code>	<code>null</code> o veces ≤ 0 → <code>IllegalArgumentException</code> . Si ya no cabe → <code>IllegalStateException</code> .
<code>calcularTotal()</code>	El for polimórfico , lo más importante que van a escribir hoy. Suma <code>items[i].calcularPrecio()</code> y aplica el descuento. Ese método está escrito por cuatro grupos distintos y ustedes no necesitan saber cuál es cuál.
<code>calcularTotal(double)</code>	El total más la propina. Propina negativa → excepción.
<code>calcularTotal(Cobable)</code>	El total más el despacho. Fíjense en el tipo del parámetro: es una interfaz, no una clase. Al pedido no le importa si reparte una moto o una bicicleta, solo exige que sepa cobrar.
<code>detalle()</code>	Texto de varias líneas: encabezado, una línea por item con <code>describir()</code> , y el total.
<code>getPedidosEmitidos()</code>	<code>static</code> : cuántos pedidos lleva el programa completo, no este objeto.
<code>aplicarDescuento(pct)</code>	Fuera de 0-100 → <code>IllegalArgumentException</code> . Devuelve el total nuevo.

PARA COMPILAR SOLO LO SUYO

```
javac Producto.java Descontable.java Cobable.java Pedido.java
```

CÓMO SABEN QUE QUEDÓ BIEN

- [OK] G3 cumple `Descontable`, contrato del G2 — sección C
- [OK] `agregar(p) + agregar(p,2)` dejan 3 items — sección E
- [OK] `calcularTotal(bici) = total + despacho` — sección E
- [OK] Cada pedido recibe un `numero` distinto — sección H

Si terminan antes: agreguen una cuarta sobrecarga, `calcularTotal(Cobable despacho, double propina)`, y expliquen en el cierre por qué el compilador nunca se confunde entre las cuatro.

clase abstracta propia 2 hijas IVA 19% emiten la boleta

Ustedes escriben la **segunda clase abstracta del sistema**, y esa es toda la gracia de su parte: van a decidir qué se escribe una sola vez en el padre y qué está obligada a responder cada hija. El banco de pruebas va a intentar hacer `new MedioPago(...)` y espera que falle.

LOS DOS CONTRATOS

USTEDES LO ESCRIBEN

Cobable

```
double montoACobrar()
```

```
boolean cobrar(double monto)
```

Está obligado a cumplirlo el **Grupo 5**, en Repartidor, y el **Grupo 3** lo usa como tipo de parámetro en una de sus sobrecargas: dos grupos dependen de ustedes. **Escríbanla** en `Cobable.java`: llega vacía, con la especificación en su Javadoc.

DEBEN CUMPLIRLO

Facturable

```
String detalleBoleta()
```

```
double totalConIVA()
```

Lo escribe el **Grupo 3**. Se implementa **una sola vez**, en el padre abstracto.

LO QUE TIENE QUE HACER CADA MÉTODO

MÉTODO	REGLA
<code>MedioPago(titular, monto)</code>	Monto negativo → <code>IllegalArgumentException</code> .
<code>abstract recargo()</code>	No lo escriben: lo declaran . Es lo único que de verdad diferencia a una tarjeta de un pago en efectivo.
<code>abstract nombreMedio()</code>	Igual: cada hija dice cómo se llama.
<code>totalConIVA()</code>	$(\text{monto} + \text{recargo}()) \times 1.19$. Fíjense: llama a <code>recargo()</code> , que en esta clase todavía no existe. Eso es polimorfismo.
<code>detalleBoleta()</code>	Una línea tipo BOLETA TARJETA 3 cuota(s) Camila neto \$10000 recargo \$600 TOTAL \$12614. Para formatear usen <code>Producto.pesos(monto)</code> .
<code>Tarjeta(titular, monto)</code>	Sobrecarga corta: una cuota.
<code>Tarjeta(..., cuotas)</code>	Solo de 1 a 12 cuotas.
<code>Tarjeta.recargo()</code>	0 si es una cuota. Si no, 2% del monto por cada cuota.
<code>Efectivo.recargo()</code>	Cero, siempre.
<code>Efectivo.vuelto()</code>	Método propio, que no existe en el padre: lo que pagó menos el total con IVA. Si diera negativo, 0.

PARA COMPILAR SOLO LO SUYO

```
javac Producto.java Facturable.java MedioPago.java Tarjeta.java Efectivo.java
```

CÓMO SABEN QUE QUEDÓ BIEN

- [OK] `MedioPago (G4)` es abstracta → `InstantiationException` — sección A
- [OK] `Tarjeta` extends `MedioPago` — sección B
- [OK] `G4` cumple `Facturable`, contrato del `G3` — sección C
- [OK] `Facturable` calcula el IVA (`10000` → `11900`) — sección C

Si terminan antes: agreguen `Transferencia` como tercera hija y cuenten en el cierre **cuántas líneas** les costó. Ese número es el argumento entero a favor de las clases abstractas.

clase abstracta propia 2 hijas super.montoACobrar() cruzan la jerarquía

Ustedes escriben la **tercera clase abstracta** y, sobre todo, son dueños de la interfaz más importante del taller. **Calificable** la implementan su Repartidor y la Pizza del Grupo 1, que no comparten ninguna clase padre. Ninguna herencia puede unir esas dos cosas; una interfaz sí. En la demo van a aparecer las dos en el mismo arreglo.

LOS DOS CONTRATOS

USTEDES LO ESCRIBEN

Calificable

```
void calificar(int estrellas)
```

```
double promedioEstrellas()
```

Está obligado a cumplirlo el **Grupo 1**, en Pizza y Sushi, y ustedes también lo implementan en Repartidor. **Escribanla** en **Calificable.java**: llega vacía, con la especificación en su Javadoc.

DEBEN CUMPLIRLO

Cobrabable

```
double montoACobrar()
```

```
boolean cobrar(double monto)
```

Lo escribe el **Grupo 4**.

LO QUE TIENE QUE HACER CADA MÉTODO

MÉTODO	REGLA
Repartidor(nombre)	Nombre vacío o <code>null</code> → <code>IllegalArgumentException</code> . Entregas en 0.
abstract tarifaBase()	Lo declaran, no lo escriben: cada vehículo cobra distinto.
abstract vehiculo()	Igual.
montoACobrar()	En el padre: la tarifa base y nada más.
cobrar(double)	Si el monto no alcanza para <code>montoACobrar()</code> , devuelve <code>false</code> y no entrega. Si alcanza: suma una entrega, suma lo recaudado y devuelve <code>true</code> .
calificar(int)	Solo de 1 a 5. Fuera de rango → <code>IllegalArgumentException</code> .
promedioEstrellas()	Suma ÷ votos, y 0 si todavía no hay ninguno.
describir()	Algo como Ignacio en MOTO (3 entregas, 4.5 estrellas).
Moto: tarifaBase()	\$2500. Vehículo: "MOTO".
Moto: montoACobrar()	Sobrescribe al padre : \$1000 más si está lloviendo. No repitan la fórmula, llamen a <code>super.montoACobrar()</code> y súmenle encima.
Bici	\$1500 y "BICICLETA". No sobrescribe <code>montoACobrar()</code> : se conforma con lo que hereda. Comparen las dos hijas, ahí está toda la herencia en dos archivos.

PARA COMPILAR SOLO LO SUYO

```
javac Cobrabable.java Calificable.java Repartidor.java RepartidorMoto.java RepartidorBici.java
```

CÓMO SABEN QUE QUEDÓ BIEN

- [OK] Repartidor (G5) es abstracta → `InstantiationException` — sección A
- [OK] G5 cumple Cobrabable, contrato del G4 — sección C
- [OK] Producto y Repartidor NO comparten clase padre — sección D
- [OK] ...pero caben en el mismo arreglo `Calificable[]` — sección D

Si terminan antes: agreguen `RepartidorAuto`, que cobra por kilómetro. Después respondan: si `Calificable` fuera una clase abstracta en vez de una interfaz, ¿podría la Pizza del Grupo 1 seguir siendo calificable? ¿Por qué no?